

Build and Demonstrate a Location Profitability Model

Final refined dbt Canvas guide for Azure Synapse, based on the exact workflow and UI behavior validated during the hands-on exercise



Validated result
The Canvas model ran successfully against Azure Synapse and was generated as a version-controlled SQL model under models/canvas.

Who	Estimated time	Output grain
Analyst / analytics engineer	30-45 minutes	One row per location

Prerequisites

- Jaffle Shop project connected to dbt Cloud and Azure Synapse.
- Production or staging job completed with Generate docs on run enabled.
- Catalog contains the locations and orders marts.
- Canvas workspace can access the production metadata and development target.
- Working Git branch and pull request workflow are available.

1. Canvas Controls Used in This Exercise

Action	How to perform the action
Open Join editor	Connect both inputs, then double-click inside the Join node where it says "Combine inputs."
Open Aggregate or Formula editor	Double-click inside the transform node.
Preview rows	Select a node and press P, or use the node preview control.
Preview columns	Select a node and press C.
Inspect generated SQL	Click SQL in the Canvas toolbar.
Name the dbt model	Edit the Output node's Model name.
Persist model code	Use Commit; Canvas creates a visual-editor branch and Git commit.
Merge into target branch	Create a pull request from Studio or GitHub and merge it.

Important terminology

In the Aggregate and Formula editors, the generated column name is entered in the field labeled Alias, not "Output Name."

Verified input columns

Model	Columns used
locations	location_id, location_name, tax_rate, opened_date
orders	order_id, location_id, order_total, order_cost

2. Build the Visual Model

1

Create or open the workspace

Create a Canvas workspace named Location profitability. Canvas will initially provide an Output node named Untitled.sql.

2

Add locations

Choose Input, search for locations, and add it. Select the node and press P to preview rows. Verify location_id, location_name, tax_rate, and opened_date.

3

Add orders

Choose Input, search for orders, and add it. Preview the node and verify order_id, location_id, order_total, and order_cost.

4

Add the Join operator

Choose Transform and add a Join. Connect locations to the L input and orders to the R input.

5

Open the Join editor

Double-click inside the Join node where it says Combine inputs. Set Join Type to Left.

6

Use the Synapse-compatible advanced expression

In Advanced expression, enter the fully qualified condition shown below. Do not use the L and R shortcut aliases.

Required join expression

```
locations.location_id = orders.location_id
```

This generates: `ON locations.location_id = orders.location_id`

Avoid this expression

```
L.location_id = R.location_id
```

In the tested Canvas/Synapse combination, this produced undefined aliases in the generated SQL.

7

Control duplicate join columns

In Join output columns, keep the locations-side location_id and remove the duplicate orders-side location_id if needed. Keep the order fields required for aggregation.

8

Preview the Join

Select Join and press P. Confirm that locations without orders remain present and display null order columns because the join is Left.

3. Configure the Aggregate

9

Add Aggregate

Add an Aggregate transform after Join and connect Join to Aggregate. Double-click inside Aggregate to open edit mode.

10

Add Group by columns

Add location_id, location_name, tax_rate, and opened_date to Group by. The target grain is one row per location.

11

Add aggregate metrics

For each metric, select the function and source column, then enter the resulting name in Alias.

Function	Source column	Alias
COUNT / COUNT_BIG	order_id	total_orders
SUM	order_total	total_revenue
SUM	order_cost	total_cost
AVG	order_total	average_order_value

12

Preview Aggregate

Select Aggregate and press P. Confirm there is one row per location and that total_orders, total_revenue, total_cost, and average_order_value are populated for active locations.

Expected visual flow

locations + orders -> Join 1 -> Aggregate 1 -> Formula 1 -> Output

4. Add Formula and Output

13

Add Formula Transform

Add a Formula Transform after Aggregate. Double-click Formula to open edit mode.

14

Create gross profit

In Formula, enter the expression below. Put the generated column name in the Alias field.

Field	Value
Expression	total_revenue - total_cost
Alias	gross_profit

15

Preview Formula

Select Formula and press P. Verify the new gross_profit column equals total_revenue minus total_cost.

16

Connect the Output node

Connect Formula to the existing Output node. The Output node is the dbt model that Canvas creates.

17

Rename the Output

Change Model name from Untitled.sql to location_profitability_canvas_v1.sql. Keep the file path as models/canvas unless your project uses another convention.

18

Add model description

Use: "Location-level profitability summary created in dbt Canvas. Includes order count, revenue, cost, average order value, and gross-profit metrics."

19

Run the model

Click Run. Confirm the run reports success and creates the renamed model in the development schema. If the prior log still says Untitled, rerun after renaming.

Validated run state

The successful run should show PASS=1, WARN=0, ERROR=0 for the Canvas-generated view.

5. Inspect SQL, Commit, and Merge

20

Inspect generated SQL

Click SQL. Verify the Join uses the fully qualified ON expression, the Aggregate contains the expected groupings and aliases, and Formula adds gross_profit.

The critical Join section should resemble:

```
FROM locations
LEFT JOIN orders
ON locations.location_id = orders.location_id
```

21

Compare with the Codex reference model

Compare models/canvas/location_profitability_canvas_v1.sql with the Codex-generated models/marts/location_profitability_v1.sql. Validate grain, metrics, readability, and Synapse compatibility.

22

Commit from Canvas

Click Commit. Use the message: Add location_profitability_canvas_v1 generated with dbt Canvas.

What Commit does

Canvas creates a Git commit on its own branch, typically named visual-editor/<workspace-id>/<session-id>. It does not merge the change into synapse-legacy or main.

23

Create a pull request

Open Studio. Under Version control, choose Create a pull request on GitHub. Set the target branch to the branch used for this Synapse project, such as synapse-legacy.

24

Review and merge

Review the generated file and SQL diff. Merge with the title: Add location_profitability_canvas_v1 generated with dbt Canvas.

6. Publish and Demonstrate the Value

25

Run the production job

After merge, run the existing Jaffle Build job from Orchestration. Use the project command below and keep Generate docs on run enabled.

```
dbt build --exclude resource_type:unit_test
```

26

Verify Catalog and lineage

Search Catalog for `location_profitability_canvas_v1`. Confirm the model, columns, description, physical relation, and upstream lineage from locations and orders.

27

Demonstrate the analyst workflow

Show that the analyst visually selected governed dbt marts, joined them, aggregated metrics, added a business formula, previewed each step, inspected SQL, and submitted the result through Git and pull request review.

Recommended demonstration story

- Governed inputs: The analyst starts from documented, production dbt marts rather than unmanaged extracts.
- Visual transformation: The analyst creates a Left Join, aggregations, and gross-profit calculation without hand-authoring the full SQL model.
- Transparency: Canvas exposes the generated SQL for technical review.
- Quality control: Preview is available at every node, and dbt runs the generated model against Synapse.
- Software delivery: Canvas writes dbt code to Git on a dedicated branch and uses a pull request for review and merge.
- Reusability: The output becomes another governed model for downstream reporting, semantic models, or additional Canvas workflows.

Suggested next Canvas model

Location executive scorecard

Use `location_profitability_canvas_v1` as the single input. Add a Formula Transform for a performance tier, select executive-facing columns, and publish `location_scorecard_canvas_v1`. This demonstrates self-service enrichment on top of a governed model without another Join.

7. Lessons Learned and Final Checklist

Lesson	Lesson Learned
Join editor	Double-click “Combine inputs” after connecting L and R.
Join expression	Use fully qualified model names for Synapse: <code>locations.location_id = orders.location_id</code> .
Aggregate naming	Use the Alias field for <code>total_orders</code> , <code>total_revenue</code> , <code>total_cost</code> , and <code>average_order_value</code> .
Calculated field	Use Formula Transform; expression and Alias are separate.
Preview	P previews rows; C previews columns.
Output	Untitled.sql is the initial output model and must be renamed.
Commit behavior	Canvas commits to a visual-editor branch; a pull request performs the merge.
Synapse unit tests	Continue excluding <code>resource_type:unit_test</code> for the sample project because of the previously observed nested-CTE issue.

Completion checklist

- Canvas model is named `location_profitability_canvas_v1.sql`.
- Run finishes with `ERROR=0`.
- Generated SQL contains `ON locations.location_id = orders.location_id`.
- Output has one row per location.
- Canvas-created file exists under `models/canvas`.
- Canvas commit exists on a visual-editor branch.
- Pull request has been reviewed and merged.
- Production job has completed with docs generation enabled.
- Catalog displays the model and upstream lineage.
- The demonstration explains analyst enablement, governance, SQL transparency, Git controls, and downstream reuse.

Final value statement

dbt Canvas enables an analyst to create a governed, reviewable, version-controlled transformation using visual operators, while retaining SQL visibility and the standard dbt deployment lifecycle.